



LLM as Judges for Coding Tasks: Robustness and Evaluation

Team Members:

**Megha Suresh, Nandana Vikraman, Avantika Tiwari, Amulya Nekkanti,
Jahnavi Krishna Kovvuri**

Group Name: NeuroNauts

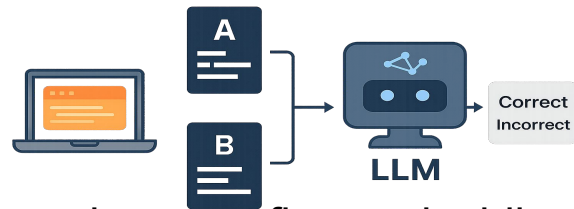
Mentor Name: Dr. Neeraj Varshney

Abstract



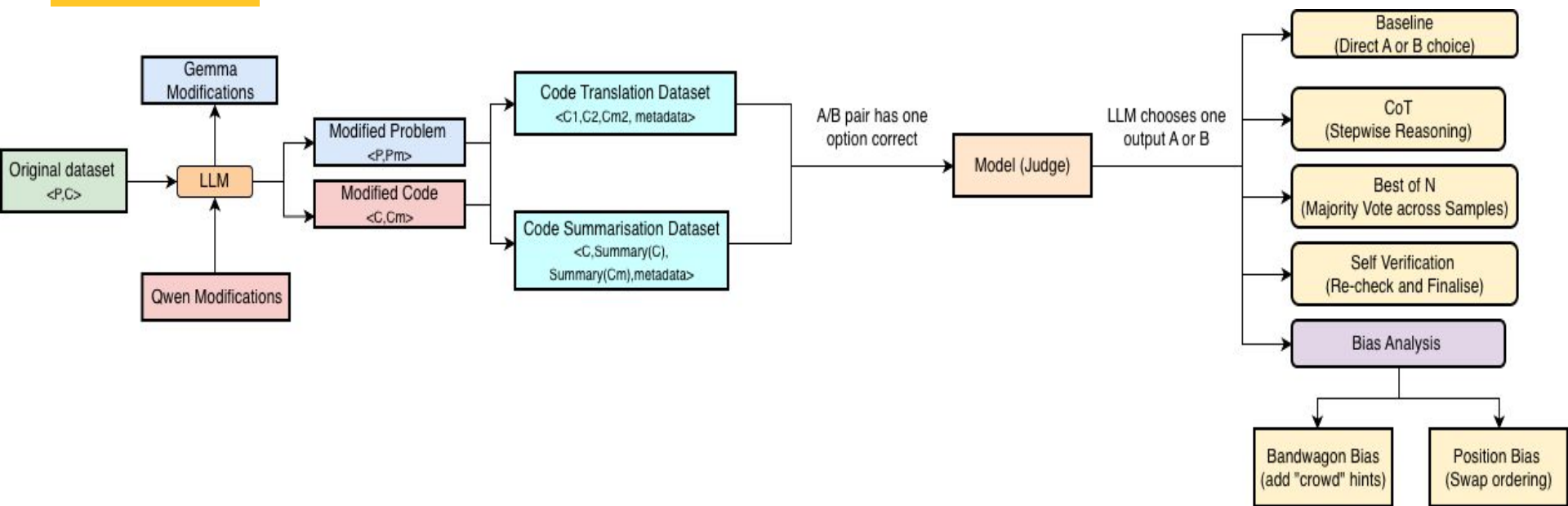
This project investigates how reliably large language models (LLMs) can act as automated judges for coding tasks, focusing on both robustness and bias. We construct a controlled A/B benchmark for code summarization and code translation by generating perturbed problem statements and buggy code variants, ensuring that each pair contains exactly one correct option. Using this setup, we evaluate four open-source models Qwen2.5, Qwen3-8B, LLaMA-8B, and Gemma-3B under multiple prompting styles, including Baseline, Chain-of-Thought, Self-Verification, and Best-of-N. Our experiments show that Qwen3-8B is the most reliable judge overall, while LLaMA-8B is consistently the weakest. Reasoning styles affect tasks differently: Self-Verification harms summarization but improves translation, and Best-of-N primarily benefits weaker models. Across all settings, the judges exhibit strong position and bandwagon biases, indicating that their decisions are highly sensitive to input framing. These findings underscore that off-the-shelf LLMs are useful but not yet robust or neutral evaluators, and our curated A/B dataset provides a foundation for fine-tuning more reliable, bias-resistant LLM judges.

Introduction



- LLMs are now used as automatic judges in software engineering workflows, deciding which summary, translation, or code solution is better.
- However, their judgments can be swayed by non-semantic factors like position bias, bandwagon bias, and unstable reasoning under CoT or self-verification.
- Most existing benchmarks only check whether the final answer is correct, not whether the judgment is robust, consistent, or truly semantics-based.
- As part of the broader LLM-as-a-Judge effort, we study four judgment tasks; our group focuses on code summarization and code translation as core tests of semantic understanding.
- We systematically evaluate Qwen2.5, Qwen3-8B, LLaMA-8B, and Gemma-3B under different prompting styles and controlled bias settings.
- This lets us see where LLM judges succeed, where they fail, and how bias shows up, motivating the need for more robust, bias-resistant LLM judges.

Method



- Gemma creates problem variations (Pm); Qwen creates code mutations (Cm).
- Variations include small rewrites, constraint shifts, and simple logic edits.
- Create Pm and Cm variants from original (P, C).
- Build summarization + translation A/B pairs.
- Evaluate models with Baseline, CoT, Best-of-N, Self-Verification.
- Test robustness via position and bandwagon biases.

Experiments

Task Reformulation

- Standard coding tasks (summarization, translation) recast as A/B judgment problems.
- Each pair has one correct and one subtly buggy candidate.
- Judge quality measured by simple accuracy over A/B pairs (semantic discrimination).

Pilot Screening: Model Landscape

- Broad pool of LLMs: **GPT family, DeepSeek, Mistral, etc.**
- Checked:
 - **Behavior as judges** across tasks.
 - **Latency & token cost** constraints.
 - **Reproducibility & openness** for research.
- Outcome: choose a **fully open-source, reproducible** evaluation stack.

Final Judge Models

- **Qwen2.5-Coder-7B-Instruct** – code-focused understanding.
- **Qwen3-8B** – strong general-purpose reasoning.
- **LLaMA-8B-Instruct** – standard open baseline.
- **Gemma-3B/4B-it** – compact instruction-tuned judge.

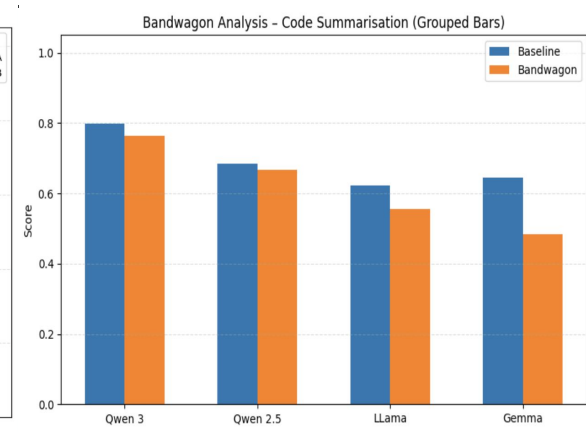
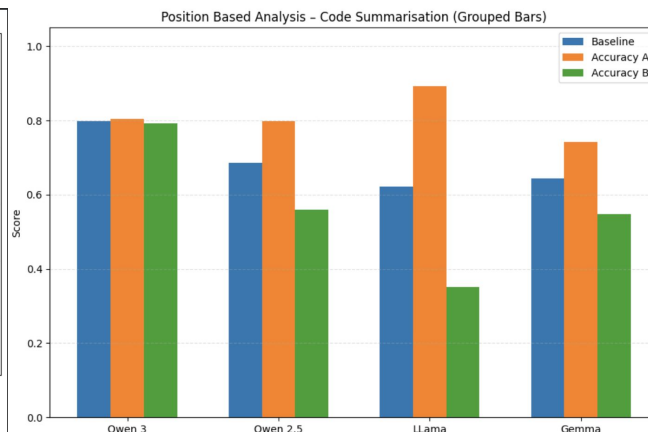
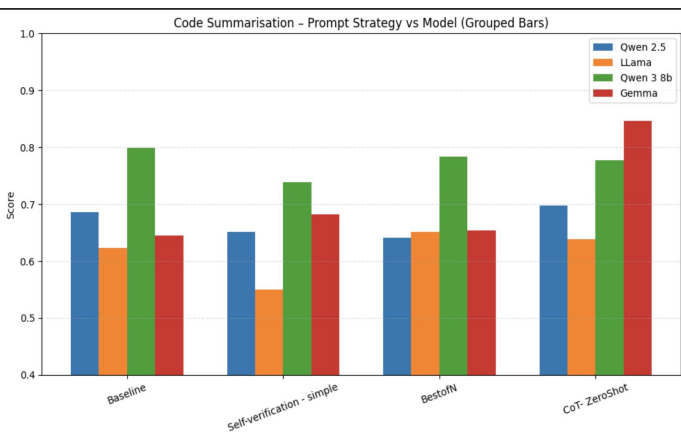
Reasoning Modes

- **Baseline** – direct A/B decision.
- **Chain-of-Thought (CoT)** – stepwise reasoning before answer.
- **Self-Verification** – re-check and finalize decision.
- **Best-of-N** – multiple samples, majority-vote judgment.
- Goal: test when “**more thinking**” helps vs. destabilizes judges.

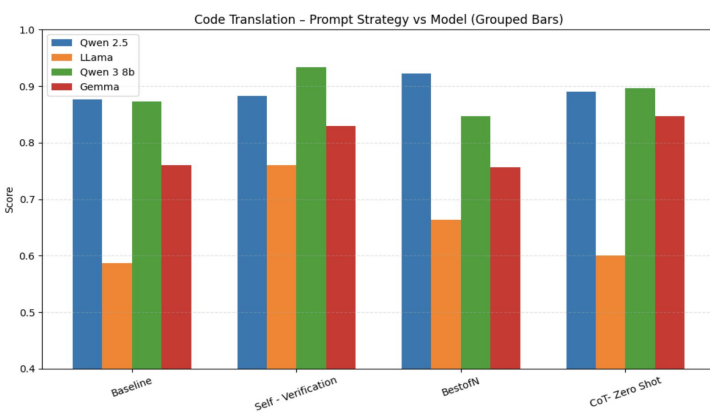
Bias Probes (Robustness Testing)

- **Position Bias** – swap A/B order, measure accuracy change / flip rate.
- **Bandwagon Bias** – add hints like “Most annotators preferred A”.
- Reveals how much judgments depend on **framing vs. content**.

Findings



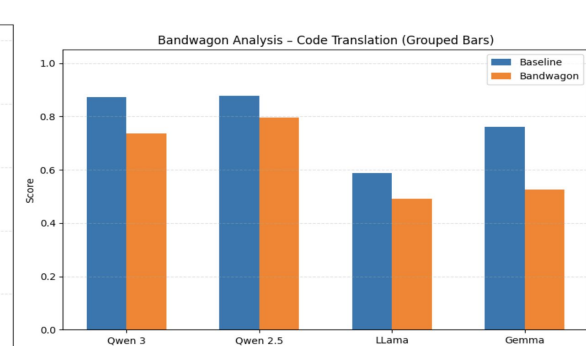
Multiple Prompting Strategy Analysis



Position Bias



Bandwagon Bias



Conclusion

- Evaluated **four open-source LLMs** as automatic judges for code summarization and translation.
- **Qwen3-8B** emerges as the most reliable judge overall, while **LLaMA-8B** is consistently the weakest.
- Prompting tweaks are **not a universal fix**: Self-Verification helps translation but hurts summarization; Best-of-N mainly benefits weaker models.

Key Takeaways

- All models show **strong position and bandwagon biases** – judgments shift with option order and “crowd” hints.
- Off-the-shelf LLMs are **useful but not yet robust or neutral** enough to be trusted as standalone judges.
- Our **A/B evaluation framework** makes these weaknesses visible and offers a structured way to analyze judge behavior.

Future Work

- Continue **detailed error analysis** of model predictions to identify systematic failure patterns.
- **Fine-tune judge models** on our curated A/B training data to improve accuracy and reduce bias.
- Re-run the full benchmark after fine-tuning (all prompting modes) to measure **gains in robustness and neutrality**.